Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Информационные системы

Лабораторная работа № 2

Импорт объектов и изоляция транзакций
Вариант 4101

Выполнил:
Чэнь Хаолинь P3316
Преподаватель:
Коновалов Арсений Антонович

г. Санкт-Петербург, 2026

Содержание

1 Задание **2**

2 Реализация **2**

2.1 Архитектура и модель данных 2

2.2 Формат импорта и проверка данных 3

2.3 Новая уникальность 3

2.4 Транзакции и обработка отказов 3

2.5 Веб-интерфейс 4

3 UML-диаграммы **4**

4 Проверка на Helios **8**

4.1 Сценарии JMeter 8

4.2 Сравнение уровней изоляции в C02 9

4.3 Функциональная проверка 9

4.4 Интерфейс приложения 10

5 Исходный код и воспроизведение **11**

6 Заключение **11**

1. Задание

Доработать ИС из ЛР1 следующим образом:

1. Добавить в систему возможность массового добавления объектов при помощи импорта файла. Формат для импорта необходимо согласовать с преподавателем. Импортируемый файл должен загружаться на сервер через интерфейс разработанного веб-приложения.

- При реализации логики импорта объектов необходимо реализовать транзакцию таким образом, чтобы в случае возникновения ошибки при импорте не был создан ни один объект.

- При импорте должна быть реализована проверка пользовательского ввода в соответствии с ограничениями предметной области из ЛР1.

- При наличии вложенных объектов в основном объекте из ЛР1 необходимо задавать значения полей вложенных объектов в той же записи, что и основной объект.

2. Необходимо добавить в систему интерфейс для отображения истории импорта (обычный пользователь видит только операции импорта, запущенные им, администратор — все операции).

- В истории должны отображаться ID операции, статус её завершения, пользователь, который её запустил, число добавленных объектов в операции (только для успешно завершённых).

3. Согласовать с преподавателем и добавить в модель из первой лабораторной новые ограничения уникальности, проверяемые на программном уровне (эти новые ограничения должны быть реализованы в рамках бизнес-логики приложения и не должны быть отображены/реализованы в БД).

4. Реализовать сценарий с использованием Apache JMeter, имитирующий одновременную работу нескольких пользователей с ИС, и проверить корректность изоляции транзакций, используемых в ЛР. По итогам исследования поведения системы при её одновременном использовании несколькими пользователями изменить уровень изоляции транзакций там, где это требуется. Обосновать изменения.

- Реализованный сценарий должен покрывать создание, редактирование, удаление и импорт объектов.

- Реализованный сценарий должен проверять корректность поведения системы при попытке нескольких пользователей обновить и/или удалить один и тот же объект (например, двух администраторов).

- Реализованный сценарий должен проверять корректность соблюдения системой ограничений уникальности предметной области при одновременной попытке нескольких пользователей создать объект с одним и тем же уникальным значением.

2. Реализация

Для импорта согласован формат JSON. Сочетание названия фильма и режиссёра проверяется в бизнес-логике.

2.1. Архитектура и модель данных

Приложение использует Jakarta EE 10: CDI управляет компонентами бизнес-логики, Jakarta REST обслуживает API, Jakarta Persistence и EclipseLink работают с PostgreSQL. Пользовательский интерфейс реализован средствами HTML, CSS и JavaScript. Транзакции имеют тип `RESOURCE_LOCAL`; их границы заданы программно через `EntityTransaction`. Аннотация `@Transactional` в данной реализации не используется.

К существующим сущностям добавлены `ImportOperation`, состояния `RUNNING`, `SUCCESS`, `FAILED` и роли `USER`, `ADMIN`. История связана с учётной записью по идентификатору. В API возвращается

безопасное представление: идентификатор операции, пользователь, статус, число добавленных фильмов только при успехе, время, имя файла и краткая причина ошибки. Пароли, соли и хеши в него не входят.

Миграция `003_lab2_roles_imports.sql` сохраняет существующие строки и назначает прежним пользователям роль USER. Повторное выполнение безопасно. Регистрация всегда создаёт обычного пользователя; роль администратора назначается серверным скриптом по точному имени уже зарегистрированной учётной записи.

2.2. Формат импорта и проверка данных

Файл передаётся на сервер как `multipart/form-data`, поле `file`, тип содержимого `application/json`. Браузер отправляет исходный объект File без преобразования JSON в JavaScript Number. Сохраняются Session Cookie и защита CSRF. Приняты ограничения 5 MiB и 1000 фильмов; пустой массив запрещён.

Верхний объект содержит целочисленный `formatVersion=1` и массив `movies`. Coordinates, Person и Location задаются вложенными объектами той же записи. Парсер проверяет UTF-8, допускает начальный BOM, отклоняет неизвестные и повторные свойства, автоматически формируемые поля, неверные типы, дробные значения целых полей, переполнение и неконечные числа. Long разбирается точно из целого JSON или десятичной строки; значение 9007199254740993 не проходит через Double. После разбора выполняется та же Bean Validation, что и при CRUD.

Для каждой записи создаются новые координаты. Человек определяется по уникальному `passportID`. Если человек уже существует, все его переданные поля должны совпадать; Location сравнивается по значениям, ZonedDateTime — с учётом часового пояса, числа — после преобразования в соответствующий Java-тип. Несовпадение вызывает `PERSON_DATA_CONFLICT`; импорт никогда не редактирует существующего человека неявно.

2.3. Новая уникальность

Бизнес-ключ имеет два независимых компонента: точное сохранённое название и `Person.id` режиссёра либо null. Регистр, пробелы и Unicode не нормализуются. Одноимённые люди с разными идентификаторами являются разными режиссёрами. Два фильма с одинаковым названием и отсутствующим режиссёром считаются дубликатами. Изменение имени или паспорта человека не меняет его идентификатор.

`MovieUniquenessValidator` используется обычным созданием, частичным обновлением и импортом. Обновление сначала объединяет входные поля с существующей сущностью и исключает собственный ID из поиска. Сравнение названия через Java String.equals обеспечивает точную семантику. Для нового сочетания в PostgreSQL отсутствуют UNIQUE, уникальные индексы и триггеры; прежние ограничения паспорта и имени пользователя сохранены.

2.4. Транзакции и обработка отказов

После проверки аутентификации, CSRF, наличия, размера и типа файла короткая транзакция T1 сохраняет RUNNING. Затем сервер разбирает файл. Транзакция T2 сначала блокирует строку `AppState(id=1)` через `PESSIMISTIC_WRITE`; все окончательные чтения, проверки версий и уникальности выполняются после получения этой блокировки. Все вложенные объекты и фильмы сохраняются в T2. SUCCESS и фактическое число фильмов фиксируются в той же транзакции; revision увеличивается один раз.

При ошибке T2 полностью откатывается. Независимая транзакция T3 сохраняет FAILED с пустым addedCount; история не исчезает вместе с объектами. Исторические транзакции не изменяют revision. Flush не является commit. Это существенно при IDENTITY: фактические INSERT

могут выполняться до конца обработки файла, но остаются частью одной транзакции.

При неопределённом исходе commit новая связь с БД сначала проверяет историю. Уже подтверждённый SUCCESS не заменяется на FAILED. При недоступности проверки операция остаётся RUNNING; клиент получает идентификатор и указание проверить историю, без автоматической повторной загрузки. После остановки прежнего единственного экземпляра скрипт запуска проверяет процесс, получает блокировку AppState и восстанавливает оставшиеся RUNNING. Этот порядок предназначен для одного экземпляра приложения.

SQLSTATE 55P03, 57014 и другие временные ошибки обрабатываются отдельно от бизнес-конфликтов. Ошибки ввода и конфликты версий возвращаются клиенту.

2.5. Веб-интерфейс

Добавлены страницы импорта и истории. Во время отправки повторный щелчок отключён; результат показывает ID операции, число фильмов либо код, номер записи и путь поля. История имеет собственный опрос, поэтому неуспешный импорт виден без изменения revision. Обычный пользователь видит свои операции, администратор — все; проверка принадлежности выполняется сервером и для списка, и для отдельной записи.

Интерфейс выполнен на русском языке. В боковой панели находятся четыре коллекции, импорт JSON, история и специальные операции. Щелчок по строке фильма открывает сведения в боковой панели или диалоговом окне, в зависимости от режима таблицы и ширины окна. Строки людей, координат и мест открывают диалоговые окна. В режиме «Все столбцы» действия находятся в первой колонке. Редактирование, удаление, закрытие окна и переходы между импортом и историей обозначены пиктограммами с текстовыми подсказками. В подробном просмотре прокручивается область полей, а заголовок и кнопка закрытия остаются на месте.

Механизм синхронизации первой работы сохранён. Открытая форма сохраняет пользовательский ввод и первоначальную версию объекта, а сервер отклоняет устаревшее сохранение.

3. UML-диаграммы

Диаграммы отражают окончательные сущности и компоненты приложения. Редактируемые исходники Graphviz DOT, изображения SVG/PNG/PDF и контрольные суммы Java-файлов находятся в каталоге `report/Lab2/uml/`.

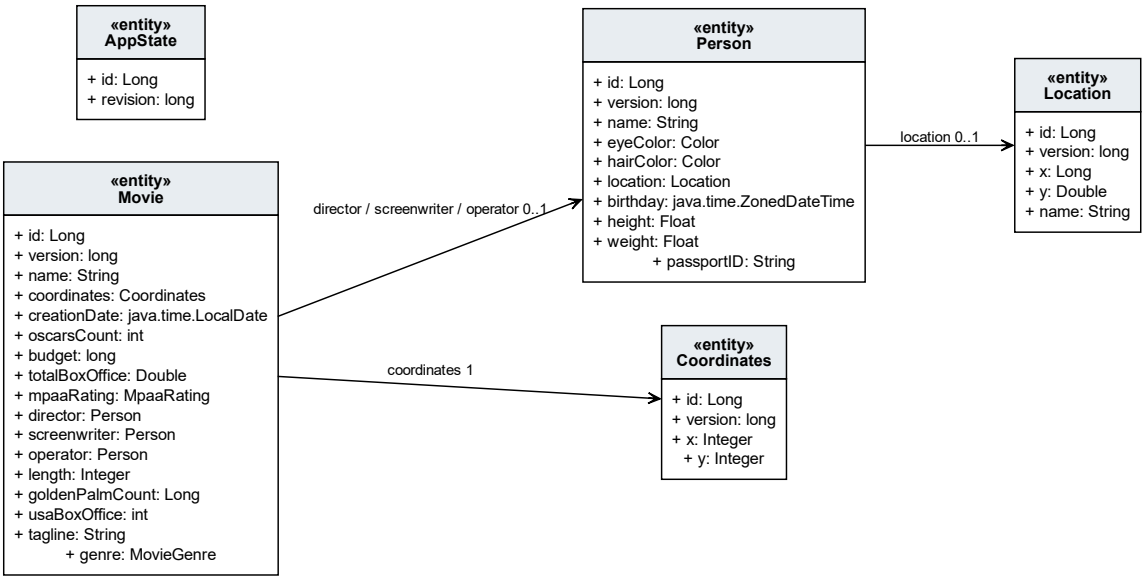


Рис. 1: Классы предметной области и связи с вложенными объектами

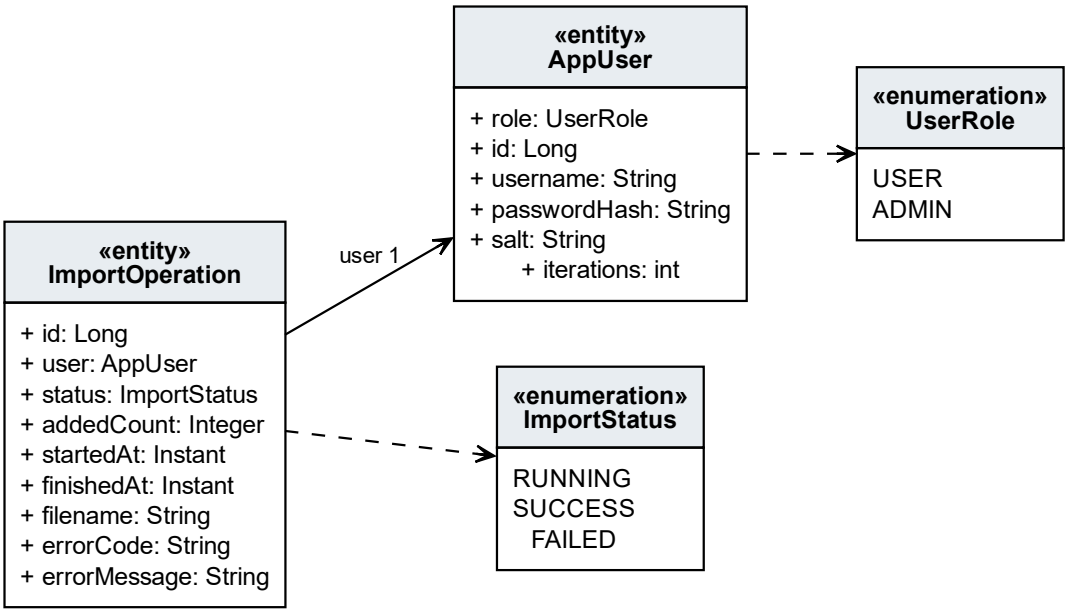


Рис. 2: Учётные записи, роли и устойчивое состояние импорта

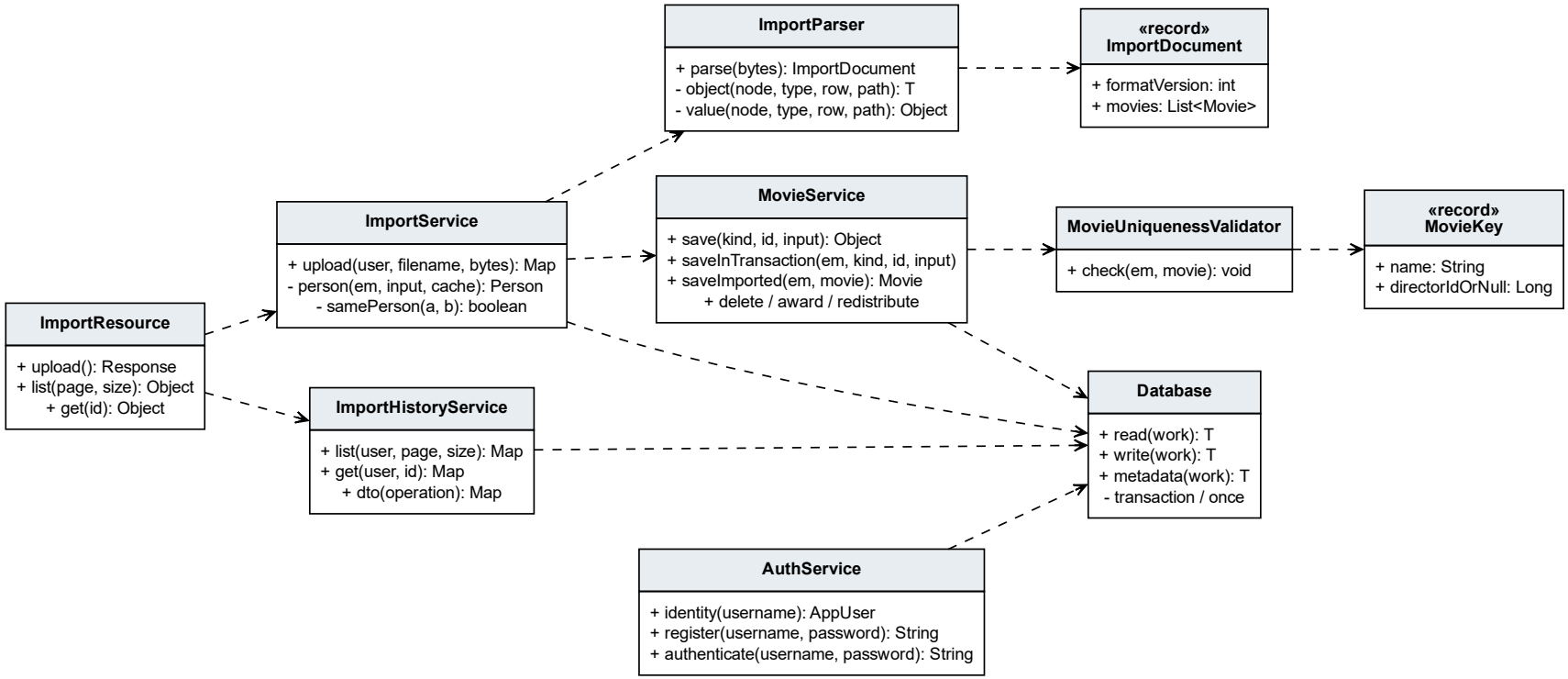


Рис. 3: Компоненты импорта, DTO и транзакционная граница

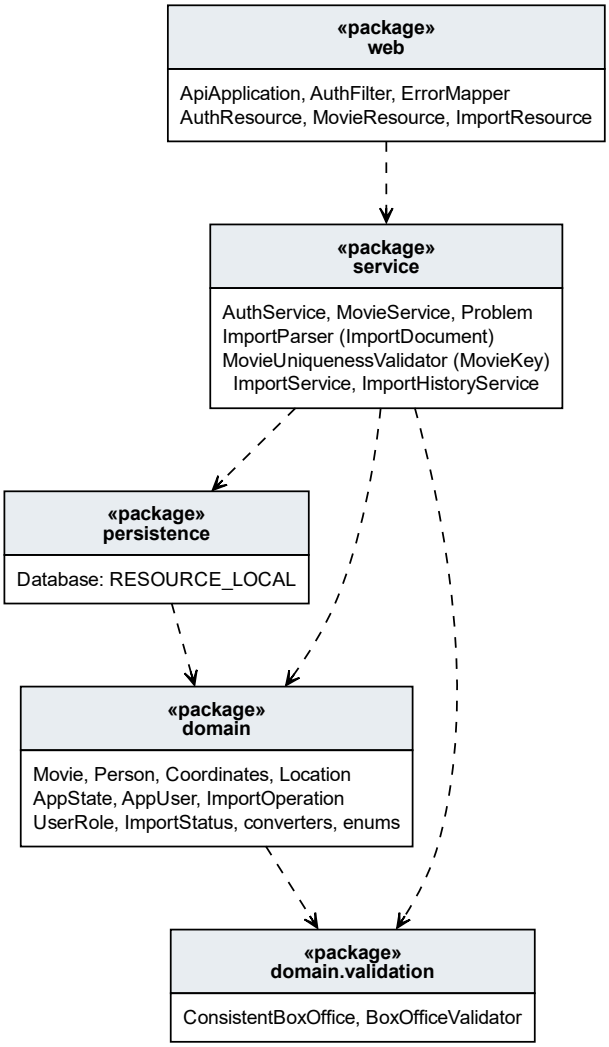


Рис. 4: Пакеты приложения и зависимости

4. Проверка на Helios

Проверка выполнена 10 ноября 2026 года. Apache JMeter 5.6.3 запускался на Helios и обращался к Payara Micro по адресу `127.0.0.1:40797`. Приложение использовало persistence unit `movies-helios-test` и таблицы PostgreSQL с префиксом `lab2_4101_test_`. Для записи применялись `READ COMMITTED`, блокировка строки AppState и проверка версии редактируемого или удаляемого объекта.

4.1. Сценарии JMeter

Сценарий	Потоки	Раунды	Запросы	Успехи	Ожидаемые отказы
C01	2	20	40	20	20
C02	10	20	200	20	180
C03	2	20	40	20	20
Всего		60	280	60	220

- **C01. Редактирование и удаление одного фильма.** Два администратора получили одинаковую версию объекта. В 9 раундах сохранилось редактирование, в 11 — удаление. При победе редактирования проверены слоган и увеличение версии на единицу. При победе удаления повторное чтение подтвердило отсутствие фильма.
- **C02. Создание фильма с общим ключом.** Десять пользователей одновременно отправили одинаковые название и идентификатор режиссёра. В каждом раунде сохранился один фильм с заданным ключом. Остальные запросы получили HTTP 409 с кодом `DUPLICATE_MOVIE`.
- **C03. Импорт файлов с общей второй записью.** Каждый файл содержит свой первый фильм и общий второй. Все 20 отклонённых файлов получили конфликт в `movies[1].director`. В каждом раунде сохранились ровно два фильма из файла-победителя. Снимки PostgreSQL подтвердили прирост на 40 фильмов, 40 координат, 20 людей и 20 мест. История сохранила успешные и отклонённые операции с соответствующими счётчиками.

Все три сценария прошли проверки HTTP-ответов и конечного состояния. Неожиданных отказов, ошибок JTL, повторяющихся ключей и остатков отклонённых импортов не обнаружено.

4.2. Сравнение уровней изоляции в C02

На Helios проведены четыре прогона C02 на тестовом экземпляре приложения с портом `40801`. Во всех прогонах использовались один WAR, блокировка AppState и десять потоков JMeter. В каждом из двадцати раундов потоки отправляли одинаковые название и идентификатор режиссёра. В копии плана задано продолжение работы после ошибки отдельного запроса. Код 503 по-прежнему считается ошибкой проверки.

Уровень изоляции задавался параметром `MOVIE_ISOLATION` при запуске приложения. В первой паре действовало ограничение по умолчанию, до трёх попыток транзакции. Во второй паре параметр `MOVIE_TX_ATTEMPTS` был равен единице.

Изоляция	Попытки	201	409	503	Повторы	Итог
`READ COMMITTED`	3	20	180	0	0	Пройден
`SERIALIZABLE`	3	20	180	0	98	Пройден
`SERIALIZABLE`	1	20	89	91	0	Ошибка
`READ COMMITTED`	1	20	180	0	0	Пройден

При трёх попытках оба уровня прошли проверку. При `SERIALIZABLE` журнал зафиксировал 98 автоматических повторов транзакций. При одной попытке 91 запрос завершился с HTTP 503 и кодом `DATABASE_TEMPORARY`. Журнал Payara показал ошибку сериализации в запросе `SELECT FOR UPDATE`. Отдельная проверка той же блокировки в PostgreSQL показала SQLSTATE `40001`. Во всех четырёх прогонах сохранялось по одному фильму за раунд, дубликатов в базе не было.

В `Database.once()` уровень изоляции устанавливается перед блокировкой AppState. Затем приложение проверяет ключ фильма, записывает объект и увеличивает счётчик AppState. При `SERIALIZABLE` ожидающая транзакция может получить ошибку сериализации, если строка AppState изменилась после получения её снимка. При `READ COMMITTED` запрос проверки выполняется после получения блокировки и видит уже зафиксированный фильм. Поэтому для проверенного пути записи выбран `READ COMMITTED` с блокировкой AppState.

4.3. Функциональная проверка

Пройдены 17 HTTP-проверок. Они охватывают вход и независимость сеансов, CSRF, ограничения multipart-загрузки, синтаксис JSON и запрет назначения роли при регистрации. Обычный пользователь получил 404 при запросе чужой операции, администратор смог открыть ту же запись. Успешный импорт двух фильмов отразился в истории со счётчиком 2. Для отклонённого импорта сохранён статус FAILED с пустым счётчиком.

В отдельной проверке файл содержал корректный первый фильм и недопустимое значение `length=0` у второго. Сервер вернул HTTP 400 с путём `movies[1].length`. Число фильмов, людей, координат и мест осталось прежним, как и версия коллекции.

4.4. Интерфейс приложения

На рисунках показан интерфейс с демонстрационными данными.

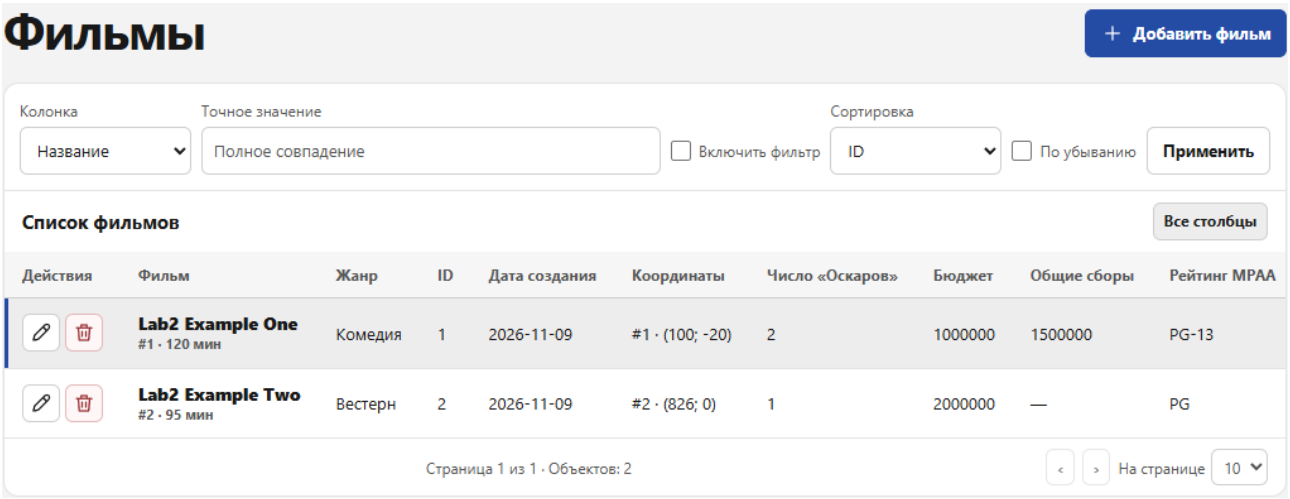


Рис. 5: Таблица фильмов в режиме «Все столбцы». Кнопки редактирования и удаления находятся в первой колонке

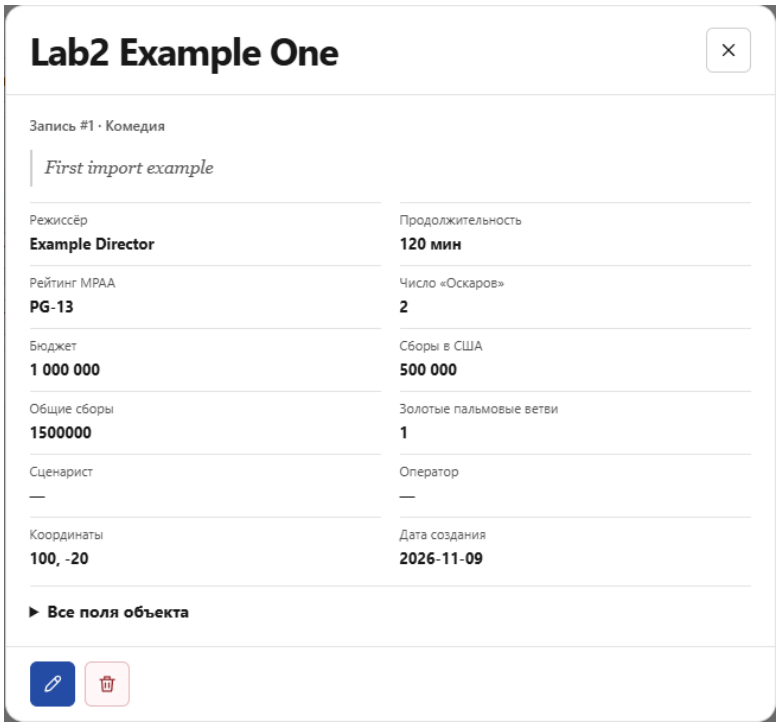


Рис. 6: Просмотр фильма с пиктограммами закрытия, редактирования и удаления

5. Исходный код и воспроизведение

Каталог `movie-lab/` содержит код, миграции, JSON-примеры и план JMeter. Доступен архив исходников. Основные отчёты JMeter и снимки PostgreSQL сохранены в `verification/Lab2/compact-20261110/`. Сравнение C02 запускается скриптом `verification/Lab2/run-isolation-c02.py`. Его результаты, HTTP-ответы и журналы находятся в `verification/Lab2/isolation-c02-20261110-221041/`.

6. Заключение

В системе реализованы атомарный JSON-импорт, история с разграничением доступа для пользователей и администраторов, а также проверка уникальности сочетания точного названия и идентификатора режиссёра в бизнес-логике.

На Helios пройдены 18 функциональных проверок и три сценария JMeter. Каждый сценарий выполнен в 20 раундах. Из 280 целевых запросов 60 завершились записью и 220 получили ожидаемый отказ. Проверены конечное состояние при конкуренции редактирования с удалением, единственная запись при совпадении ключа и откат первой записи проигравшего файла при конфликте во второй.

При `READ COMMITTED` блокировка AppState упорядочивает проверку и запись, а версия защищает от устаревшего редактирования или удаления. В основных сценариях неожиданных отказов, дубликатов и остатков отклонённых импортов не обнаружено. В отдельном сравнении C02 с одной попыткой транзакции переход с `SERIALIZABLE` на `READ COMMITTED` устранил 91 ответ HTTP 503, сохранив по одной новой записи в каждом раунде.